

Festival

Найра на фестивале играет в игру, главный приз которой — поездка в Лагуна-Колорадо. Игра заключается в использовании монет для покупки купонов. После покупки купона оставшееся количество монет может увеличиться. Цель игры — получить как можно больше купонов.

В начале игры у Найры есть A монет. Есть N купонов, пронумерованных от 0 до $N - 1$. Чтобы купить купон i , Найра должна заплатить $P[i]$ монет ($0 \leq i < N$) (при этом перед покупкой у нее должно быть не менее $P[i]$ монет). Найра может купить каждый купон не более одного раза.

Более того, каждому купону i ($0 \leq i < N$) назначен **тип** $T[i]$, который представляет собой целое число **от 1 до 4 включительно**. После того как Найра покупает купон i , оставшееся у нее количество монет умножается на $T[i]$. Формально, если в какой-то момент игры у нее есть X монет, и она покупает купон i (для этого должно выполняться $X \geq P[i]$), то после покупки у нее будет $(X - P[i]) \cdot T[i]$ монет.

Ваша задача — определить, какие купоны и в каком порядке должна купить Найра, чтобы максимизировать общее количество **купонов** у нее будет в конце игры. Если существует более одной подходящей последовательности покупок, вы можете вывести любую из них.

Детали реализации

Вы должны реализовать следующую функцию:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : начальное количество монет у Найры.
- P : массив длины N , в котором указаны цены купонов.
- T : массив длины N , в котором указаны типы купонов.
- Эта функция вызывается ровно один раз для каждого теста.

Функция должна возвращать массив R , в котором указаны покупки Найры следующим образом:

- Длина массива R должна быть равна максимальному количеству купонов, которые она может купить.
- Элементы массива — это индексы купонов, которые Найра должна купить, в хронологическом порядке. Сначала она покупает купон $R[0]$, затем купон $R[1]$ и т.д.
- Все элементы массива R должны быть уникальными.

Если ни один купон не может быть куплен, массив R должен быть пустым.

Ограничения

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ для каждого i , такого что $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ для каждого i , такого что $0 \leq i < N$.

Подзадачи и баллы

Подзадача	Баллы	Дополнительные ограничения
1	5	$T[i] = 1$ для каждого i , такого что $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ для каждого i , такого что $0 \leq i < N$.
3	12	$T[i] \leq 2$ для каждого i , такого что $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Найра может купить все N купонов (в некотором порядке).
6	16	$(A - P[i]) \cdot T[i] < A$ для каждого i , такого что $0 \leq i < N$.
7	18	Нет дополнительных ограничений.

Примеры

Пример 1

Рассмотрим следующий вызов:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Изначально у Найры есть $A = 13$ монет. Она может купить 3 купона в следующем порядке:

Куплен купон	Цена купона	Тип купона	Количество монет после покупки
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

В этом примере Найра не может купить больше 3 купонов, и описанная выше последовательность покупок — единственный способ, с помощью которого Найра может купить 3 купона. Следовательно, функция должна вернуть $[2, 3, 0]$.

Пример 2

Рассмотрим следующий вызов:

```
max_coupons(9, [6, 5], [2, 3])
```

В этом примере Найра может купить оба купона в любом порядке. Следовательно, функция должна вернуть либо $[0, 1]$, либо $[1, 0]$.

Пример 3

Рассмотрим следующий вызов:

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

В данном примере у Найры есть одна монета, которой недостаточно для покупки купонов. Следовательно, функция должна вернуть $[]$ (пустой массив).

Пример грейдера

Формат ввода:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Формат вывода:

```
S
R[0] R[1] ... R[S-1]
```

Здесь S — длина массива R , возвращаемого `max_coupons`.